

List Interface

- The **List interface** in Java is part of the `java.util` package and **extends the Collection interface**, which means it inherits all the methods of the **Collection** interface.
- It represents an **ordered collection (sequence)** of elements, allowing **duplicate values** and providing **positional access** (index-based operations).

Relationship with Parent (Collection Interface):

- List is a **child interface** of the **Collection** interface.
- It inherits general collection methods such as `add()`, `remove()`, `clear()`, and `size()`.
- It **adds its own methods** to handle elements based on their **position**, like `get(int index)`, `set(int index, E element)`, `add(int index, E element)`, and `remove(int index)`.

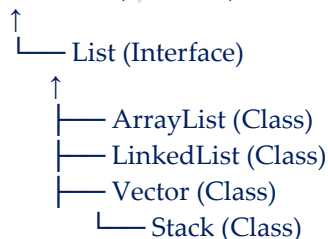
Syntax:

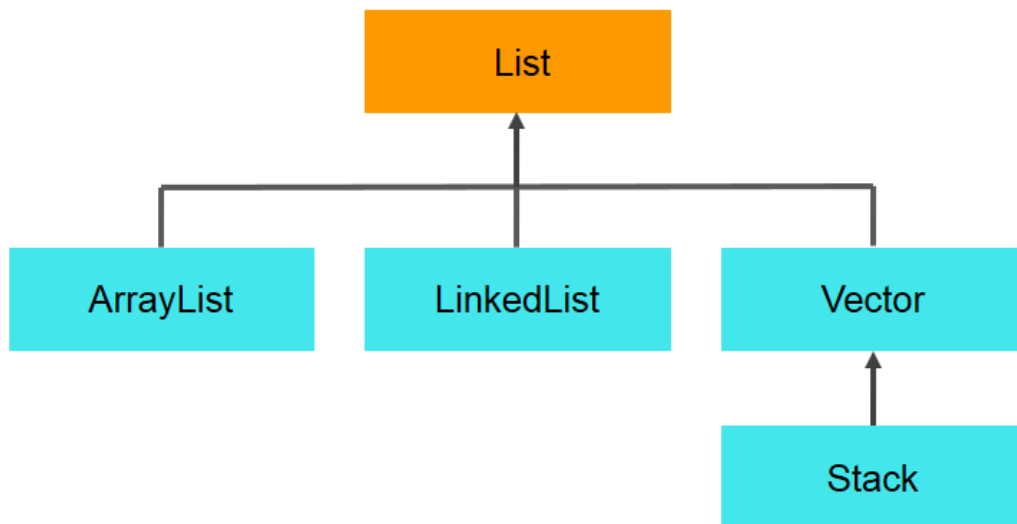
`public interface List<E> extends Collection<E>`

- `E` → Type of elements stored in the List.
- Introduced in **JDK 1.2**, as part of the Java Collections Framework.

Hierarchy of List Interface

Collection (Interface)





Properties of List Interface (Detailed Explanation)

1. Index-Based Data Structure

- Each element in a List has a specific **index** (starting from 0).
- You can access elements directly using their index.

```
List<String> fruits = new ArrayList<>();  
fruits.add("Apple"); // index 0  
fruits.add("Mango"); // index 1  
System.out.println(fruits.get(1));
```

// Output: Mango

2. Can Store Heterogeneous Data (with non-generic List)

- When generics are not used, a List can store **different data types**:

```
List list = new ArrayList();  
list.add("Hello");  
list.add(100);  
list.add(10.5);
```

- However, with **generics**, it's type-safe (only one type allowed):

```
List<String> names = new ArrayList<>();  
names.add("Aniket"); // ✓
```

```
names.add(100);    // ✗ Compile-time error
```

3. Allows Duplicate Elements

- Lists can contain **duplicate values** because they maintain order.

```
List<Integer> nums = new ArrayList<>();  
nums.add(10);  
nums.add(10);  
System.out.println(nums);  
  
// Output: [10, 10]
```

4. Allows Multiple Null Values

- You can store **any number of null elements** in a List.

```
List<String> names = new ArrayList<>();  
names.add(null);  
names.add(null);  
System.out.println(names);  
  
// Output: [null, null]
```

5. Maintains Insertion Order

- The List keeps elements **in the same order** as inserted.

```
List<String> animals = new ArrayList<>();  
animals.add("Dog");  
animals.add("Cat");  
animals.add("Tiger");  
System.out.println(animals);  
  
// Output: [Dog, Cat, Tiger]
```

6. Does Not Follow Sorting Order by Default

- Elements are stored **as inserted**, not sorted automatically.
- You can sort them manually using `Collections.sort()`.

```
List<Integer> numbers = new ArrayList<>();  
numbers.add(30);  
numbers.add(10);
```

```
numbers.add(20);
System.out.println(numbers); // [30, 10, 20]
Collections.sort(numbers);
System.out.println(numbers); // [10, 20, 30]
```

7. Allows Random Access (in some implementations)

- Classes like **ArrayList** allow **fast random access** to elements using index. But **LinkedList** doesn't — it requires traversal to reach a position.

Example:

```
ArrayList<Integer> list = new ArrayList<>();
list.add(100);
list.add(200);
System.out.println(list.get(1)); // Fast access
```

8. Supports Iterators

- You can traverse elements using:
 - Iterator
 - ListIterator (which allows **both forward and backward** traversal)

```
List<String> cities = new ArrayList<>();
cities.add("Pune");
cities.add("Mumbai");
```

```
ListIterator<String> itr = cities.listIterator();
while (itr.hasNext())
    System.out.println(itr.next());
```

9. Supports Positional Insertion and Removal

- You can **insert**, **replace**, or **remove** elements at specific positions.

```
List<String> names = new ArrayList<>();
names.add("A");
names.add("C");
names.add(1, "B"); // Insert at index 1
System.out.println(names);
```

```
// [A, B, C]
```

10. Can Contain Objects Only

- List stores **object references**, not primitive data types.
(Primitives like int, double are stored as wrapper classes Integer, Double.)

```
List<Integer> nums = new ArrayList<>(); // Wrapper class
```

11. Can Be Converted to Array and Vice Versa

- You can easily convert between a **List** and an **Array**.

```
List<String> list = Arrays.asList("A", "B", "C");  
String[] arr = list.toArray(new String[0]);
```

12. Thread-Safety Depends on Implementation

- ArrayList and LinkedList are **not synchronized**.
 - Vector and Stack are **synchronized (thread-safe)**.
-

13. Supports Functional Operations (since Java 8)

- You can use **lambda expressions**, **streams**, and **forEach()**.

```
List<Integer> list = Arrays.asList(10, 20, 30);  
list.forEach(System.out::println);
```

✓ In short, List Interface = Ordered, Indexed, Duplicates Allowed, Flexible, and Powerful for dynamic data storage.

1 Methods inherited from Collection

- `add(E e)` – Add element to the list.
 - `addAll(Collection<? extends E> c)` – Add all elements from another collection.
 - `clear()` – Remove all elements.
 - `contains(Object o)` – Check if element exists.
 - `containsAll(Collection<?> c)` – Check if all elements exist.
 - `equals(Object o)` – Compare two collections.
 - `hashCode()` – Hash code of the list.
 - `isEmpty()` – Check if empty.
 - `iterator()` – Returns an iterator.
 - `remove(Object o)` – Remove first occurrence of element.
 - `removeAll(Collection<?> c)` – Remove all elements present in another collection.
 - `retainAll(Collection<?> c)` – Retain only elements present in another collection.
 - `size()` – Number of elements.
 - `toArray()` / `toArray(T[] a)` – Convert list to array.
-

2 Methods specific to List

- `void add(int index, E element)` – Add element at a specific position.
 - `boolean addAll(int index, Collection<? extends E> c)` – Add all elements at specific index.
 - `E get(int index)` – Get element at index.
 - `E set(int index, E element)` – Replace element at index.
 - `E remove(int index)` – Remove element at index.
 - `int indexOf(Object o)` – First index of element.
 - `int lastIndexOf(Object o)` – Last index of element.
 - `ListIterator<E> listIterator()` – Returns a bidirectional iterator.
 - `ListIterator<E> listIterator(int index)` – List iterator starting at index.
 - `List<E> subList(int fromIndex, int toIndex)` – Return a portion of the list.
 - `boolean equals(Object o)` – Checks equality of lists.
 - `int hashCode()` – Returns hash code.
-

3 Iteration helpers (via Java 8+)

- `forEach(Consumer<? super E> action)` – Iterates using lambda.
- `replaceAll(UnaryOperator<E> operator)` – Replace each element using a function.
- `sort(Comparator<? super E> c)` – Sort the list.

✓ Summary

- **Indexed access:** get, set, add(index), remove(index).
 - **Search:** indexOf, lastIndexOf, contains.
 - **Bulk operations:** addAll, removeAll, retainAll.
 - **Iteration:** iterator, listIterator, forEach.
 - **Sublist:** subList to extract part of the list.
-

Here's a **complete example** demonstrating various operations on a List:

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

public class ListExample {
    public static void main(String[] args) {
        // 1. Using ArrayList
        List<String> fruits = new ArrayList<>();

        // Adding elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");
        fruits.add("Mango");
        fruits.add("Apple"); // duplicate allowed

        // Accessing elements
        System.out.println("First fruit: " + fruits.get(0));

        // Iterating using for-each loop
        System.out.println("All fruits:");
        for (String fruit : fruits) {
            System.out.println(fruit);
        }

        // Removing element
        fruits.remove("Banana");
        System.out.println("After removing Banana: " + fruits);
    }
}
```

```

// Checking if list contains an element
if (fruits.contains("Mango")) {
    System.out.println("Mango is in the list");
}

// Getting size
System.out.println("Number of fruits: " + fruits.size());

// 2. Using LinkedList
List<Integer> numbers = new LinkedList<>();
numbers.add(10);
numbers.add(20);
numbers.add(30);
numbers.add(40);

// Iterating using lambda
System.out.println("Numbers:");
numbers.forEach(n -> System.out.println(n));

// Removing by index
numbers.remove(2); // removes element at index 2 (30)
System.out.println("After removing index 2: " + numbers);

// Adding at specific position
numbers.add(1, 25);
System.out.println("After adding 25 at index 1: " + numbers);

// Sorting the list
numbers.sort(Integer::compareTo);
System.out.println("Sorted numbers: " + numbers);

// Clearing list
numbers.clear();
System.out.println("Numbers after clear(): " + numbers);
}
}

```

Answer :

First fruit: Apple

All fruits:

Apple

Banana

Orange

Mango

Apple

After removing Banana: [Apple, Orange, Mango, Apple]

Mango is in the list

Number of fruits: 4

Numbers:

10

20

30

40

After removing index 2: [10, 20, 40]

After adding 25 at index 1: [10, 25, 20, 40]

Sorted numbers: [10, 20, 25, 40]

Numbers after clear(): []